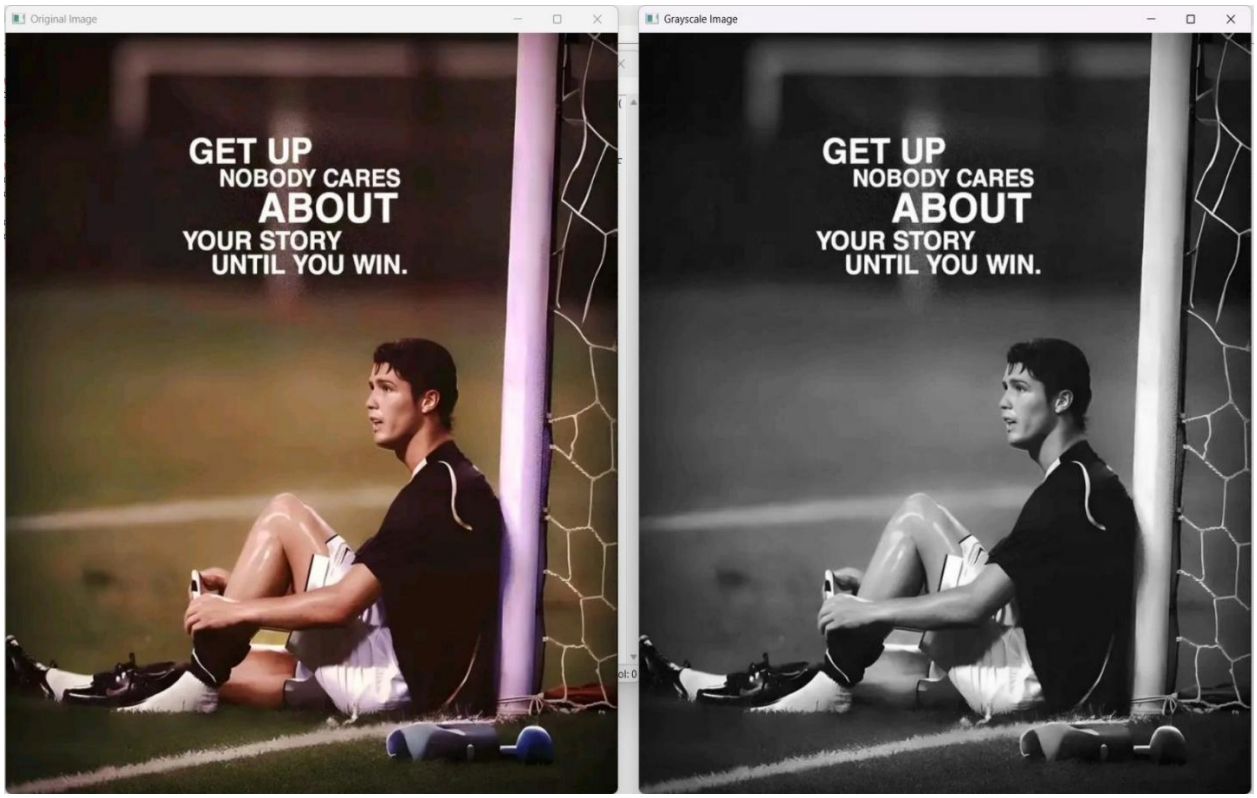
1.Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to Gray-scale.

CODE :

```
import cv2

img = cv2.imread('image.jpg')

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Display images

cv2.imshow("Original Image", img)

cv2.imshow("Gray Scale Image", gray)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

OUTPUT :

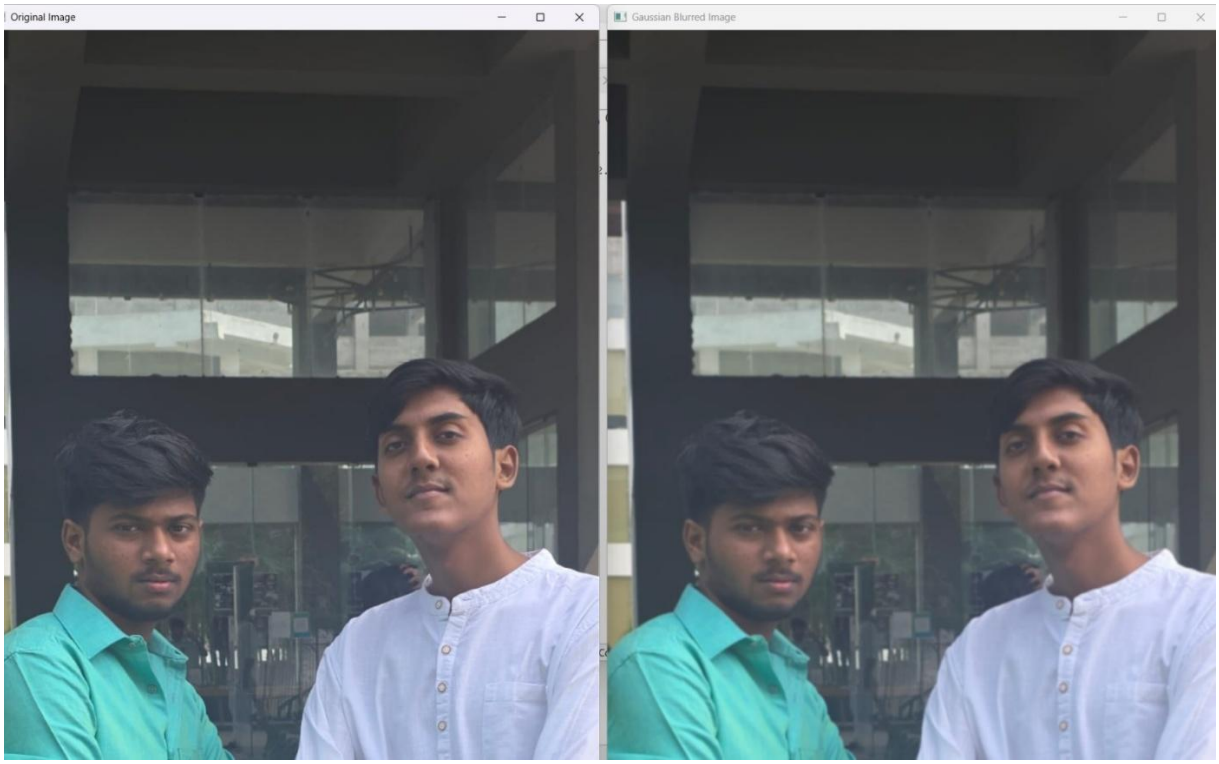


2. Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to Blur using Gaussian Blur

CODE :

```
import cv2

# Read the image

img = cv2.imread('image.jpg')

# Apply Gaussian Blur

blur = cv2.GaussianBlur(img, (5,5), 0)

# Display images

cv2.imshow("Original Image", img)

cv2.imshow("Gaussian Blurred Image", blur)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

OUPUT :

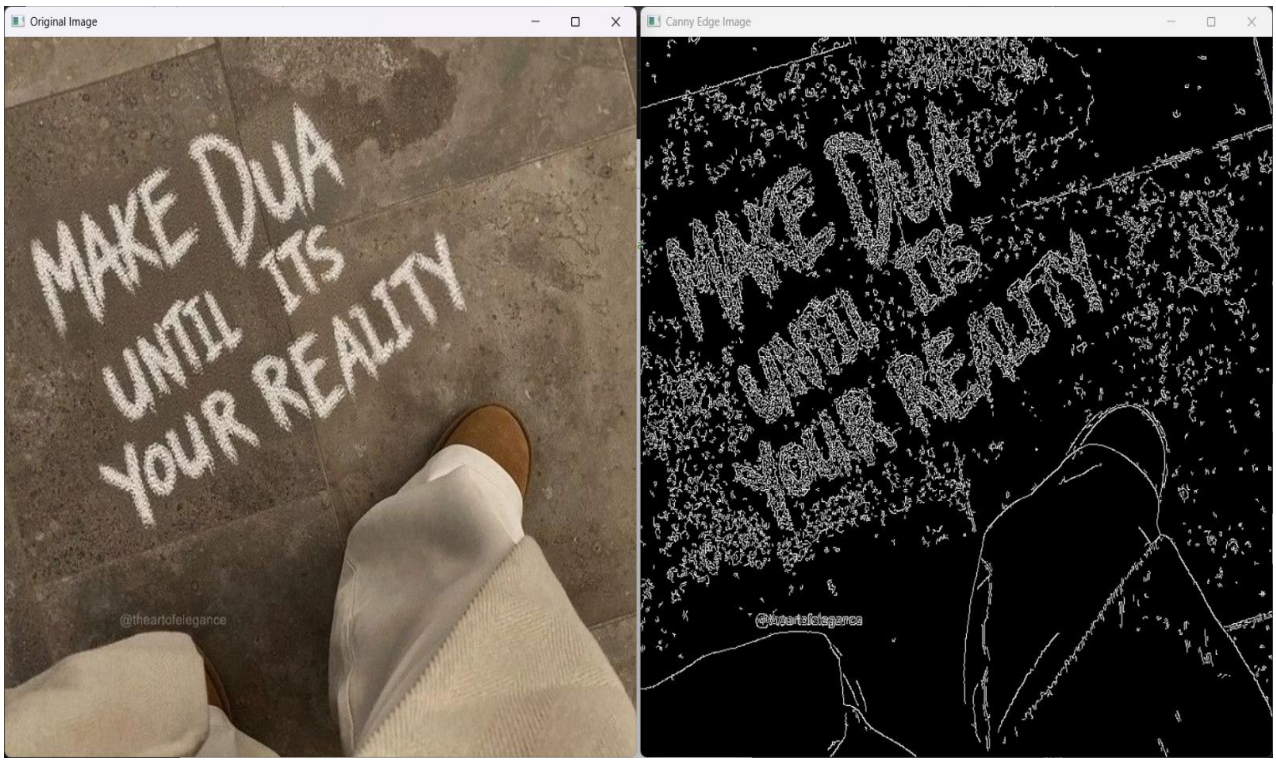


3.Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to show outline using Canny function.

CODE :

```
import cv2

# Read the image

img = cv2.imread('image.jpg')

# Convert to grayscale

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Apply Canny Edge Detection

edges = cv2.Canny(gray, 100, 200)

# Display images

cv2.imshow("Original Image", img)

cv2.imshow("Canny Edge Outline", edges)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

CODE :

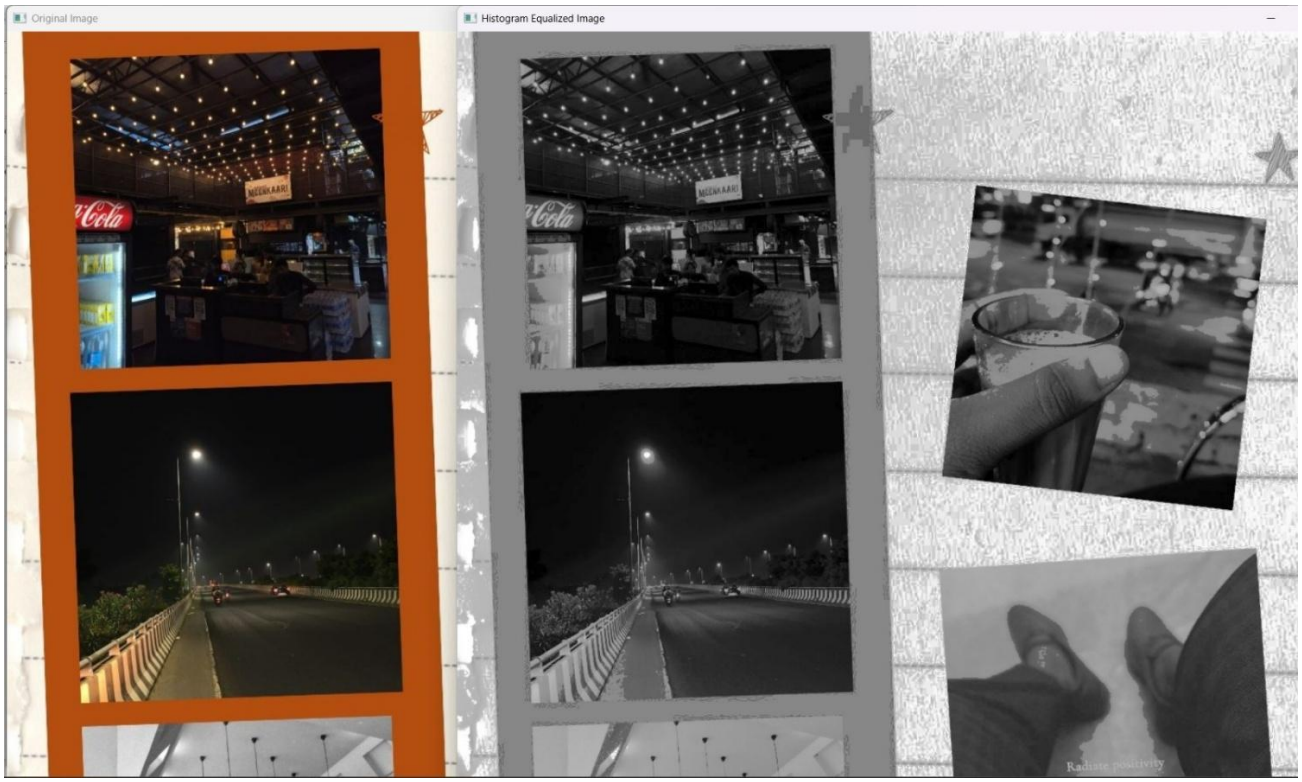


4.Implement histogram equalization on the given image and compare it with the original image using Open CV.

CODE :

```
import cv2

# Read image in grayscale

img = cv2.imread('image.jpg', 0)

# Apply Histogram Equalization

equalized = cv2.equalizeHist(img)

# Display images

cv2.imshow("Original Image", img)

cv2.imshow("Histogram Equalized Image", equalized)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

OUTPUT :

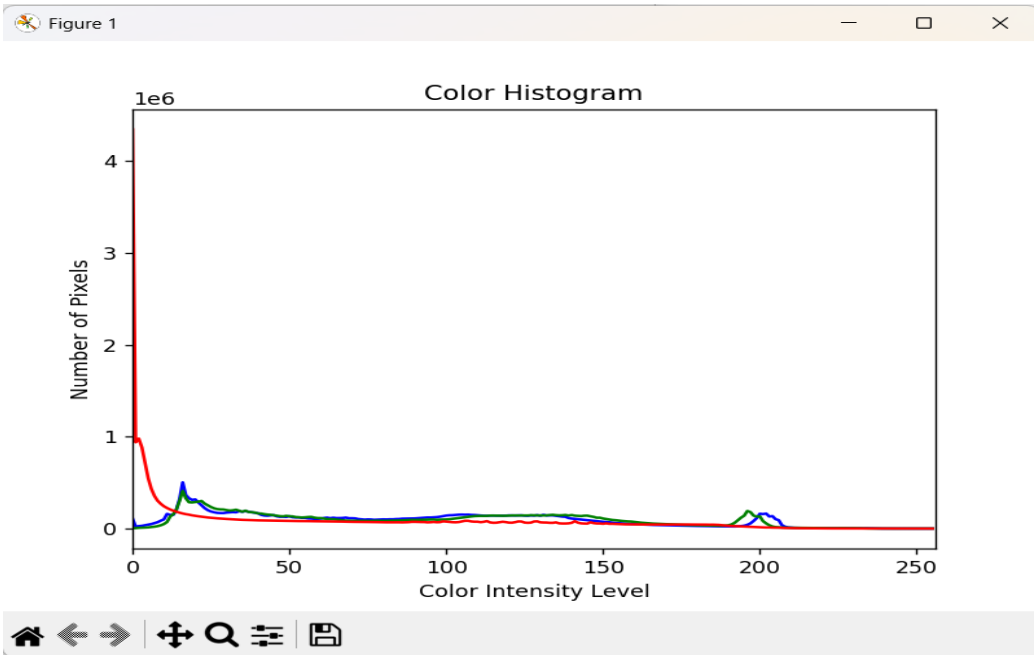


5.Write a Python function to analyze the histogram of the given input image based on color levels using Open CV.

CODE :

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

# Read the image

img = cv2.imread('image.jpg')

# Split the image into B, G, R channels

colors = ('b','g','r')

for i, col in enumerate(colors):

   hist = cv2.calcHist([img],[i],None,[256],[0,256])

   plt.plot(hist, color=col)

plt.title("Color Histogram")

plt.xlabel("Pixel Intensity")

plt.ylabel("Frequency")

plt.show()
```

OUTPUT :

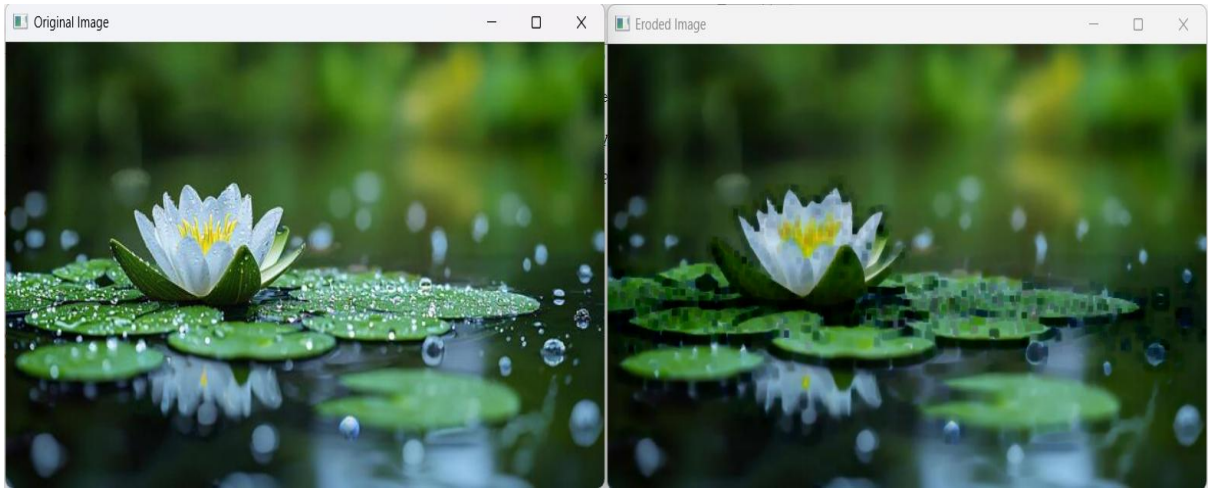


6.Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to erode using Erode function.

CODE :

```
import cv2

import numpy as np

image = cv2.imread("C:/Users/lalit/Downloads/nature.jpg")

# Step 2: Check if image loaded properly

if image is None:

   print("Error: Image not found!")

   exit()

kernel = np.ones((5,5), np.uint8)

eroded_image = cv2.erode(image, kernel, iterations=1)

cv2.imshow("Original Image", image)

cv2.imshow("Eroded Image", eroded_image)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

OUTPUT :



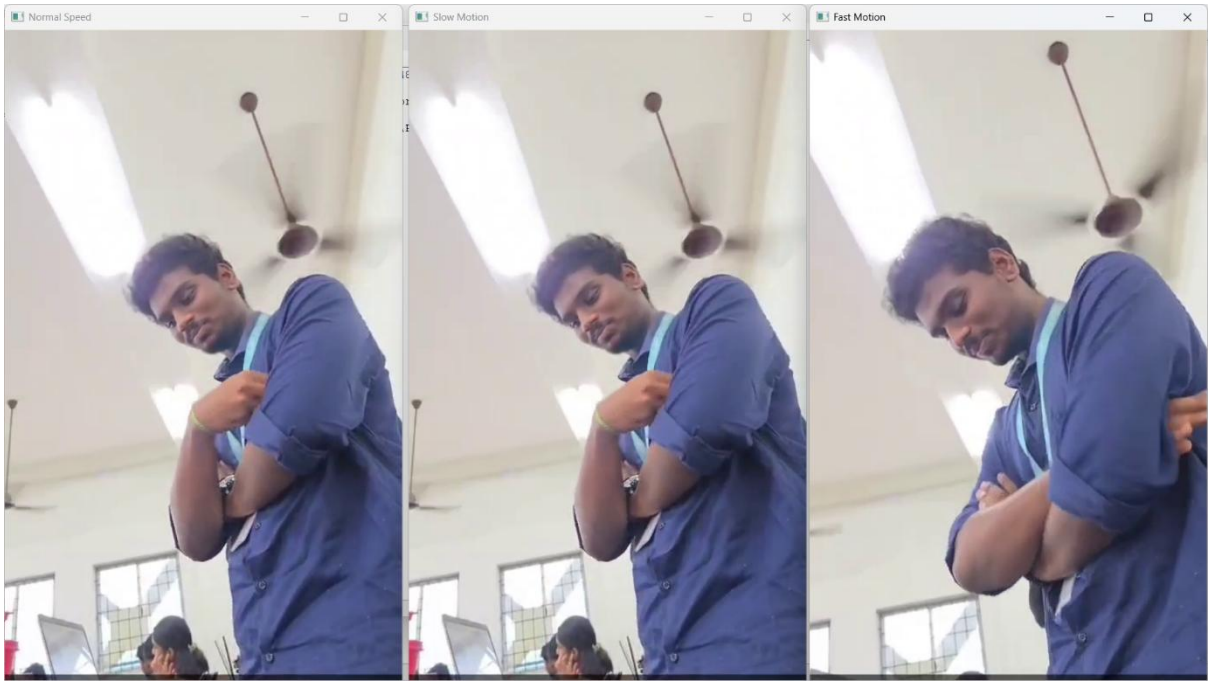
7.Perform basic video processing operations on the captured video. Read captured video in python and display the video, in slow motion and in fast motion.

CODE :

```
import cv2


cap_normal = cv2.VideoCapture("C:/Users/lalit/Downloads/video.mp4")

cap_slow = cv2.VideoCapture("C:/Users/lalit/Downloads/video.mp4")

cap_fast = cv2.VideoCapture("C:/Users/lalit/Downloads/video.mp4")

if not cap_normal.isOpened():

    print("Error opening video")

    exit()


fps = cap_normal.get(cv2.CAP_PROP_FPS)

normal_delay = int(1000 / fps)

slow_delay = int(1000 / (fps / 2))   # Half speed

fast_delay = int(1000 / (fps * 2))   # Double speed

while True:

    ret1, frame1 = cap_normal.read()

    ret2, frame2 = cap_slow.read()

    ret3, frame3 = cap_fast.read()

    if not ret1 or not ret2 or not ret3:

        break

    # Fast → skip one extra frame

    cap_fast.read()

    cv2.imshow("Normal Speed", frame1)

    cv2.imshow("Slow Motion", frame2)

    cv2.imshow("Fast Motion", frame3)
```

```
    # Control delays
    cv2.waitKey(1)
    cv2.waitKey(slow_delay)
    cv2.waitKey(1)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap_normal.release()
cap_slow.release()
cap_fast.release()
cv2.destroyAllWindows()
```

OUTPUT :

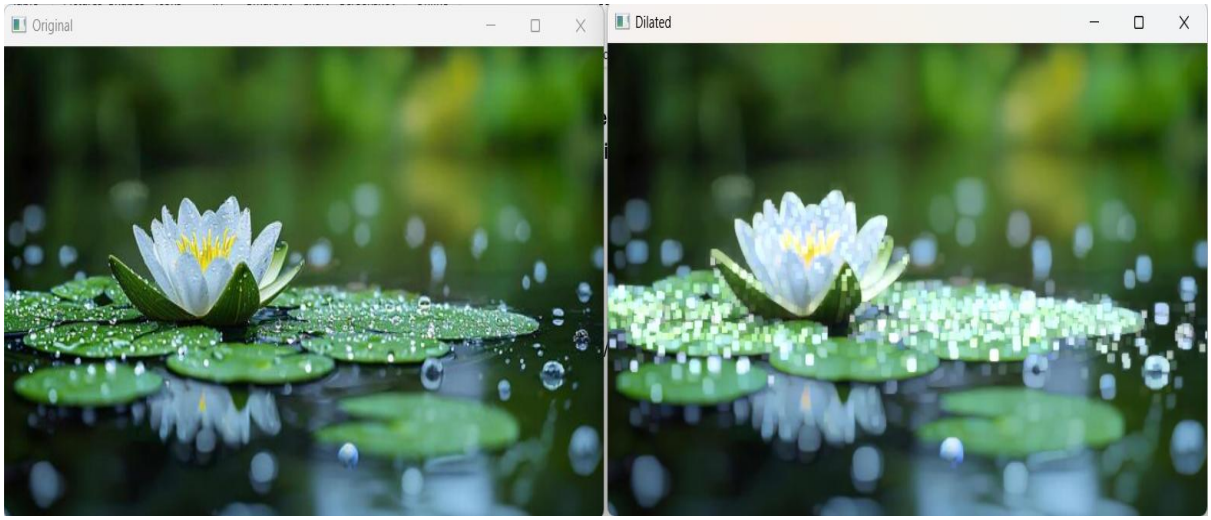


8.Perform basic Image Handling and processing operations on the image is to read an image in python and Dilate an Image using Dilate function

CODE :

```
import cv2

import numpy as np

img = cv2.imread("C:/Users/lalit/Downloads/nature.jpg")

kernel = np.ones((5,5), np.uint8)

dilated = cv2.dilate(img, kernel, iterations=1)

cv2.imshow("Original", img)

cv2.imshow("Dilated", dilated)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

OUTPUT :

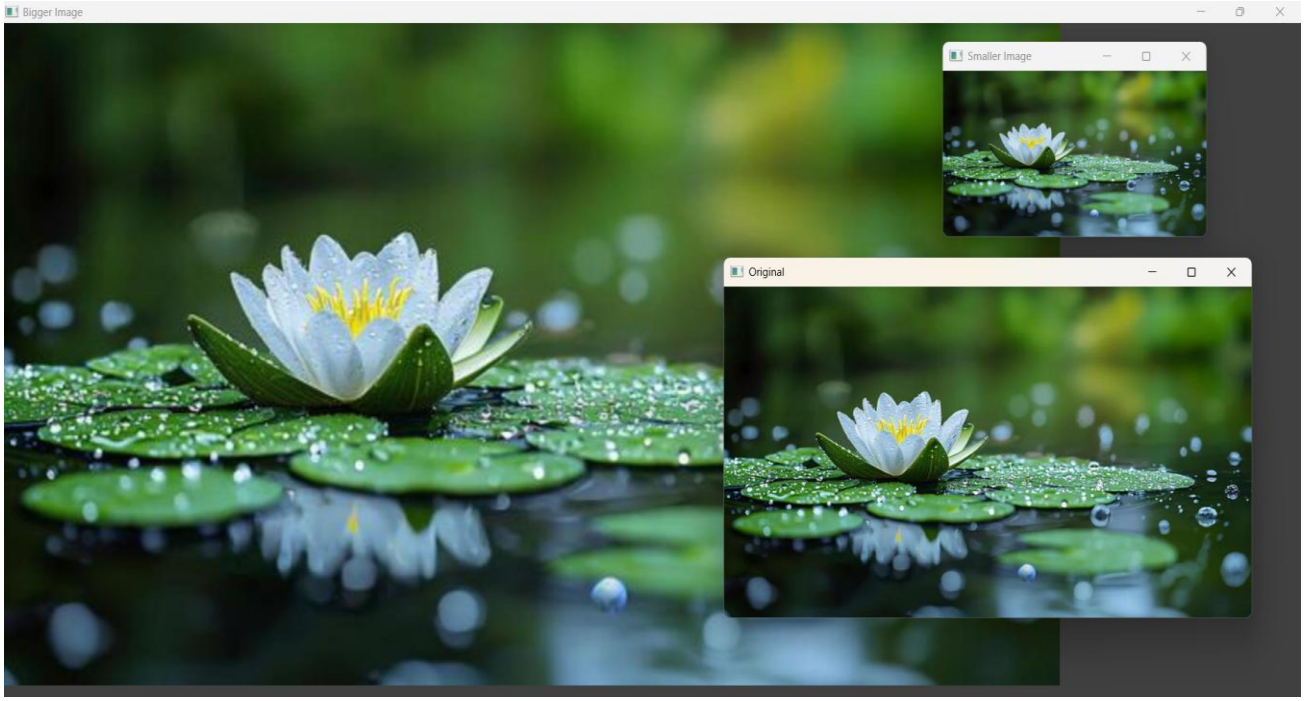


9.Implement the image scaling techniques to resize the images to both bigger and small sizes

CODE :

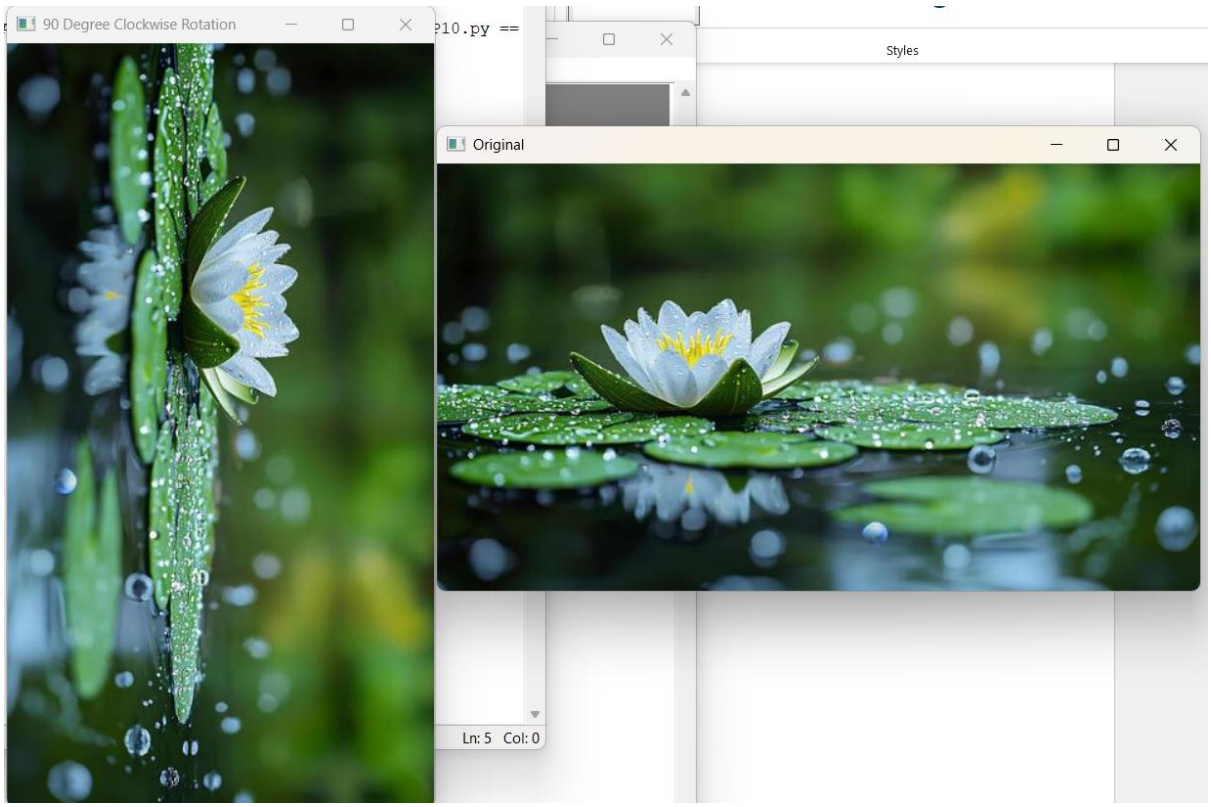
```
import cv2

img = cv2.imread("C:/Users/lalit/Downloads/nature.jpg")

# Smaller size

small = cv2.resize(img, None, fx=0.5, fy=0.5)

# Bigger size

big = cv2.resize(img, None, fx=2, fy=2)

cv2.imshow("Original", img)

cv2.imshow("Smaller Image", small)

cv2.imshow("Bigger Image", big)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

OUTPUT :



10.Perform a 90-degree rotation clockwise along the y-axis for the given image.

CODE :

```
import cv2

img = cv2.imread("C:/Users/lalit/Downloads/nature.jpg")

rotated = cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)

cv2.imshow("Original", img)

cv2.imshow("90 Degree Clockwise Rotation", rotated)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

OUTPUT :